

Assignment 9.3

2303A51250

L.Indu

Batch – 18

Task Description #1: Basic Docstring Generation

Scenario

You are developing a utility function that processes numerical lists and must be properly documented for future maintenance.

Requirements

- Write a Python function to return the sum of even numbers and sum of odd numbers in a given list
- Manually add a Google Style docstring to the function
- Use an AI-assisted tool (Copilot / Cursor AI) to generate a function-level docstring
- Compare the AI-generated docstring with the manually written docstring
- Analyze clarity, correctness, and completeness

Expected Output

- Python function with manual Google-style docstring
- AI-generated docstring for the same function
- Comparison explaining differences between manual and AI-generated documentation
- Improved understanding of AI-generated function-level documentation

Prompt :

#1.Task 1: Basic Docstring Generation

#Generate a Google-style function-level docstring for the following Python function that calculates the sum of even and odd numbers in a list.

#The docstring should describe parameters, return values, and any possible errors.

#Compare the AI-generated docstring with the manually written docstring provided below.

Manually written docstring for comparison

#printing the docstrings for comparison

```
#1.Task 1: Basic Docstring Generation
#Generate a Google-style function-level docstring for the following Python function that calculates the sum of even and odd numbers in a list.
#The docstring should describe parameters, return values, and any possible errors.
#Compare the AI-generated docstring with the manually written docstring provided below.
def sum_even_odd(numbers):
    """
    Calculates the sum of even and odd numbers in a list.

    Args:
        numbers (list): A list of integers.

    Returns:
        tuple: A tuple containing the sum of even numbers and the sum of odd numbers.

    Raises:
        TypeError: If the input is not a list or if any element in the list is not an integer.
        ValueError: If the list is empty.
    """
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list.")

    if not numbers:
        raise ValueError("Input list cannot be empty.")

    even_sum = 0
    odd_sum = 0

    for num in numbers:
        if not isinstance(num, int):
            raise TypeError("All elements in the list must be integers.")

        if num % 2 == 0:
            even_sum += num
        else:
            odd_sum += num

    return even_sum, odd_sum
```

```

        even_sum += num
    else:
        odd_sum += num

    return even_sum, odd_sum
# Manually written docstring for comparison
def sum_even_odd(numbers):
    """
    This function takes a list of integers and calculates the sum of even and odd numbers separately.

    Parameters:
    numbers (list): A list of integers to be processed.

    Returns:
    tuple: A tuple containing two elements:
        - The first element is the sum of even numbers.
        - The second element is the sum of odd numbers.

    Raises:
    TypeError: If the input is not a list or if any element in the list is not an integer.
    ValueError: If the input list is empty.
    """
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list.")

    if not numbers:
        raise ValueError("Input list cannot be empty.")

    even_sum = 0
    odd_sum = 0

    for num in numbers:
        if not isinstance(num, int):
            raise TypeError("All elements in the list must be integers.")

        if num % 2 == 0:
            even_sum += num

```

```

> Assignment_9.py > ...
39 def sum_even_odd(numbers):
57
58     if num % 2 == 0:
59         even_sum += num
60     else:
61         odd_sum += num
62
63     return even_sum, odd_sum
64 #printing the docstrings for comparison
65 print("AI-generated docstring:")
66 print(sum_even_odd.__doc__)
67 print("\nManually written docstring:")
68 print(sum_even_odd.__doc__)
69
70

```

Output

```
PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/A
AI-generated docstring:

This function takes a list of integers and calculates the sum of even and odd numbers separately.

Parameters:
numbers (list): A list of integers to be processed.

Returns:
tuple: A tuple containing two elements:
    - The first element is the sum of even numbers.
    - The second element is the sum of odd numbers.

Raises:
TypeError: If the input is not a list or if any element in the list is not an integer.
ValueError: If the input list is empty.

Manually written docstring:
Manually written docstring:

This function takes a list of integers and calculates the sum of even and odd numbers separately.

Manually written docstring:
Manually written docstring:
Manually written docstring:
Manually written docstring:

This function takes a list of integers and calculates the sum of even and odd numbers separately.

Parameters:
numbers (list): A list of integers to be processed.

Returns:
tuple: A tuple containing two elements:
    - The first element is the sum of even numbers.
    - The second element is the sum of odd numbers.

Raises:
TypeError: If the input is not a list or if any element in the list is not an integer.
ValueError: If the input list is empty.
```

Analysis :

The manual docstring is more detailed and complete. It clearly describes the input type, return type, possible errors, and provides an example. This makes the function easier to understand and maintain.

This task helped me understand that AI can assist in generating documentation quickly, but manual refinement improves clarity and completeness.

Task Description #2 :

Automatic Inline Comments

Scenario

You are developing a student management module that must be easy to understand for new developers.

Requirements

- Write a Python program for an `sru_student` class with the following:
 - Attributes: `name`, `roll_no`, `hostel_status`
 - Methods: `fee_update()` and `display_details()`
- Manually write inline comments for each line or logical block
- Use an AI-assisted tool to automatically add inline comments
- Compare manual comments with AI-generated comments
- Identify missing, redundant, or incorrect AI comments

Expected Output

- Python class with manually written inline comments
- AI-generated inline comments added to the same code
- Comparative analysis of manual vs AI comments
- Critical discussion on strengths and limitations of AI-generated comments

Prompt :

##task2.automatic inline comment generation

#Add meaningful inline comments to the following Python class.

#The comments should explain each attribute, method, and logical block clearly so that new developers can easily understand the code.

Constructor to initialize student details

Method to update student fee

Method to display student details

Code:

```
##task2.automatic inline comment generation
#Add meaningful inline comments to the following Python class.
#The comments should explain each attribute, method, and logical block clearly so that new developers can easily understand the code.
class SRUStudent:

    # Constructor to initialize student details
    def __init__(self, name, roll_no, hostel_status):
        self.name = name # Store student name
        self.roll_no = roll_no # Store student roll number
        self.hostel_status = hostel_status # Store hostel status (True/False)
        self.fee_balance = 0 # Initialize fee balance to zero

    # Method to update student fee
    def fee_update(self, amount):
        if amount < 0: # Check if amount is negative
            print("Invalid fee amount") # Display error message
        else:
            self.fee_balance += amount # Add amount to fee balance

    # Method to display student details
    def display_details(self):
        print("Name:", self.name) # Print student name
        print("Roll No:", self.roll_no) # Print roll number
        print("Hostel Status:", self.hostel_status) # Print hostel status
        print("Fee Balance:", self.fee_balance) # Print fee balance

#ai generated inline comments for comparison
class SRUStudent:

    def __init__(self, name, roll_no, hostel_status):
        # Initialize student attributes
        self.name = name
        self.roll_no = roll_no
```

```

Assignment_9.py > ...
106 class SRUStudent:
107     def __init__(self, name, roll_no, hostel_status):
108         self.name = name
109         self.roll_no = roll_no
110         self.hostel_status = hostel_status
111         self.fee_balance = 0 # Default fee balance
112
113     def fee_update(self, amount):
114         # Update the fee balance
115         if amount < 0:
116             print("Invalid fee amount")
117         else:
118             self.fee_balance += amount
119
120     def display_details(self):
121         # Display all student information
122         print("Name:", self.name)
123         print("Roll No:", self.roll_no)
124         print("Hostel Status:", self.hostel_status)
125         print("Fee Balance:", self.fee_balance)
126
127 #output from user input
128 student1 = SRUStudent("Alice", "12345", True)
129 student1.fee_update(500)
130 student1.display_details()
131 #comparison of inline comments
132 print("\nAI-generated inline comments:")
133 print(SRUStudent.__doc__)
134 print("\nManually written inline comments:")
135 print(SRUStudent.__doc__)
136

```

Output:

```

PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/ANIL/Desktop/AI Assisant/Assignment_9.py"
Name: Alice
Roll No: 12345
Hostel Status: True
Fee Balance: 500

AI-generated inline comments:
None

Manually written inline comments:
None
PS C:\Users\ANIL\Desktop\AI Assisant>

```

Analysis:

- 1.Manual comments provide deeper understanding.
- 2.AI comments are useful but sometimes lack detail.

3. Combining AI-generated comments with manual refinement gives the best result.

4. This task improved understanding of code documentation quality.

Task Description #3

Module-Level and Function-Level Documentation

Scenario

You are building a small calculator module that will be shared across multiple projects and

requires structured documentation.

Requirements

- Write a Python script containing 3–4 functions (e.g., add, subtract, multiply, divide)
- Manually write NumPy Style docstrings for each function
- Use AI assistance to generate:
 - A module-level docstring
 - Individual function-level docstrings
- Compare AI-generated docstrings with manually written ones
- Evaluate documentation structure, accuracy, and readability

Expected Output

- Python script with manual NumPy-style docstrings
- AI-generated module-level and function-level documentation
- Comparison between AI-generated and manual documentation
- Clear understanding of structured documentation for multi-function scripts

Additional Requirement

- Push the complete project documentation as a .md file to a GitHub repository
- Ensure documentation covers module overview and function descriptions

Prompt :

##task3.Module-Level and Function-Level Documentation

##write a python program to build a small calculator module that will be shared across multiple projects.

#it should have the following functions: add(), subtract(), multiply(), divide()

#Manually write NumPy Style docstrings for each function.

```

##task3.Module-Level and Function-Level Documentation
##write a python program to build a small calculator module that will be shared across multiple projects.
##it should have the following functions: add(), subtract(), multiply(), divide()
##Manually write NumPy Style docstrings for each function.

def add(a, b):
    """
    Compute the sum of two numbers.

    Parameters
    -----
    a : float
        The first number.
    b : float
        The second number.

    Returns
    -----
    float
        The sum of a and b.
    """
    return a + b

def subtract(a, b):
    """
    Compute the difference of two numbers.

    Parameters
    -----
    a : float
        The first number.
    b : float

```

```

Assignment_9.py > ...
61 def subtract(a, b):
62     """
63     Compute the difference of two numbers.
64
65     Parameters
66     -----
67     a : float
68         The first number.
69     b : float
70         The second number.
71
72     Returns
73     -----
74     float
75         The difference of a and b.
76     """
77     return a - b
78
79 def multiply(a, b):
80     """
81     Compute the product of two numbers.
82
83     Parameters
84     -----
85     a : float
86         The first number.
87     b : float
88         The second number.
89
90     Returns
91     -----
92     float
93         The product of a and b.
94     """
95     return a * b
96
97 def divide(a, b):

```

```

def divide(a, b):
    """
    Compute the quotient of two numbers.

    Parameters
    -----
    a : float
        The dividend.
    b : float
        The divisor.

    Returns
    -----
    float or str
        The quotient of a and b, or an error message if division by zero occurs.
    """
    if b == 0:
        return "Error: Division by zero is not allowed."
    return a / b

# Example usage
num1 = 10
num2 = 5
print(f"Addition: {add(num1, num2)}")
print(f"Subtraction: {subtract(num1, num2)}")
print(f"Multiplication: {multiply(num1, num2)}")
print(f"Division: {divide(num1, num2)}")
# Compare manual docstrings and AI-generated docstrings for clarity, correctness, and completeness. Explain differences.
def docstring_comparison():
    print("\n--- Docstring Comparison ---")

    print("\nClarity:")
    print("Manual docstrings are clear and concise.")

```

```

Assignment_9.py > ...
15 |         return a / b
16 |
17 | # Example usage
18 | num1 = 10
19 | num2 = 5
20 | print(f"Addition: {add(num1, num2)}")
21 | print(f"Subtraction: {subtract(num1, num2)}")
22 | print(f"Multiplication: {multiply(num1, num2)}")
23 | print(f"Division: {divide(num1, num2)}")
24 | # Compare manual docstrings and AI-generated docstrings for clarity, correctness, and completeness. Explain differences.
25 | def docstring_comparison():
26 |     print("\n--- Docstring Comparison ---")
27 |
28 |     print("\nClarity:")
29 |     print("Manual docstrings are clear and concise.")
30 |     print("AI-generated docstrings provide more detailed explanations.")
31 |
32 |     print("\nCorrectness:")
33 |     print("Both sets of docstrings correctly describe the parameters, return values, and exceptions.")
34 |
35 |     print("\nCompleteness:")
36 |     print("AI-generated docstrings include additional information about the function's behavior and edge cases.")
37 |     print("Manual docstrings focus on the basic functionality without extra details.")
38 |
39 |     print("\nConclusion:")
40 |     print("AI-generated documentation is more comprehensive, while manual documentation is straightforward and to the point.")
41 | if __name__ == "__main__":
42 |     docstring_comparison() # Run the docstring comparison analysis

```

Output:

```
PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/ANIL/Desktop/AI Assisant/Assignment_9.py"
Addition: 15
Subtraction: 5
Multiplication: 50
Division: None

--- Docstring Comparison ---

Clarity:
Manual docstrings are clear and concise.
AI-generated docstrings provide more detailed explanations.

Correctness:
Both sets of docstrings correctly describe the parameters, return values, and exceptions.

Completeness:
AI-generated docstrings include additional information about the function's behavior and edge cases.
Manual docstrings focus on the basic functionality without extra details.

Conclusion:
AI-generated documentation is more comprehensive, while manual documentation is straightforward and to the point.
PS C:\Users\ANIL\Desktop\AI Assisant>
```

Analysis:

1. Difference between module-level and function-level documentation
2. AI-generated vs manual documentation comparison
3. Professional documentation for reusable modules

